Introduzione alle Coroutines

"It is rather difficult to find short, simple examples of coroutines which illustrate the importance of the idea; the most useful coroutine applications generally are quite lengthy"
Knuth [The Art of Computer Programming]

```
git clone https://bitbucket.org/michele_dionisio/workshop_coroutine.git

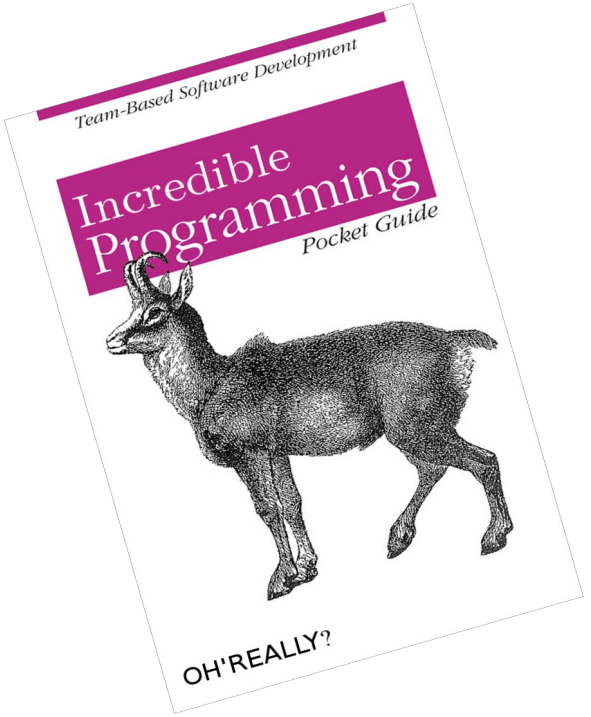
cd workshop_coroutine/

git submodule init

git submodule update
```

```
apt-get install libevent-dev cmake
```

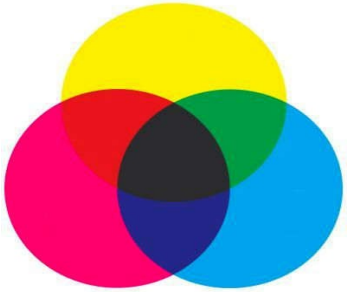
- E' da qualche anno che, scherzosamente, ai colleghi ho parlato di “Incredible Programming”. Con questo termine mi riferivo a programmi scritti in modo astrusissimo di difficilissima manutenzione praticamente immodificabili ed incomprensibili se non conoscendo la storia “umana” che esisteva dietro ogni singola riga di codice.



- Oggi vorrei farvi notare come possa capitare che affrontando i problemi in modo diverso si possa scrivere codice più semplice da capire e mantenere.
- Se entro la fine vi avrò invogliato a cercare sempre soluzioni semplici spero di avere l'onore di poter leggere codice scritto da voi in futuro *(Se per caso mi trovassi nella situazione di doverlo fixare, sicuramente vi ringrazierei)*.

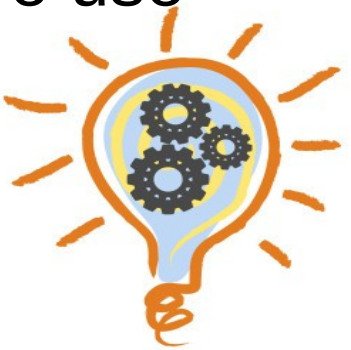
Parleremo quindi di

- **Coroutines** – un concetto poco utilizzato e conosciuto anche in linguaggi che lo implementano nativamente
- **Concorrenza** – uno negli argomenti maggiormente trattati in informatica ma dove è sempre possibile trovare il maggior numero di problemi
- Mescoliamo il tutto ...



Come procederemo

- Problema produttore consumatore e soluzione cooperativa
- Guardiamo le glibc (*rapidamente senza farsi prendere paura*).
- Estendiamo la collaborazione. Non solo due entità cooperanti. Vediamo uno scheduler collaborativo (in C)
- Vedremo che l'idea non è male!! e vedremo come possiamo usarla assieme a librerie per la programmazione event-driven per rendere il loro uso semplice e mantenibile anche in C.



Produttore - Consumatore

- Produttore

```
def produce():
  """
  Function that produce a new data
  """
  Pass
  # send data

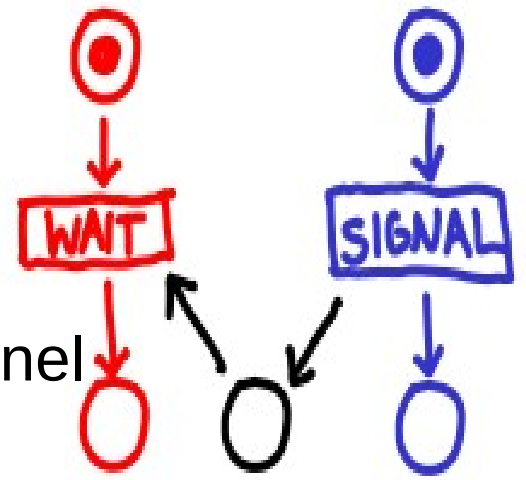
while True:
  produce()
```

- Consumatore

```
def getData():
  """
  Get data
  """
  Data = 0  # receive data
  return data

while True:
  data = getData()
  # do something
```

- Spesso per far lavorare il produttore ed il consumatore il programmatore sceglie di
  - Scrivere il codice del consumatore innestato nel produttore (o viceversa)
  - Far girare le due attività in due thread/task separati inventandosi un qualche sistema per farli comunicare. Memoria condivisa/queue/pipe/...

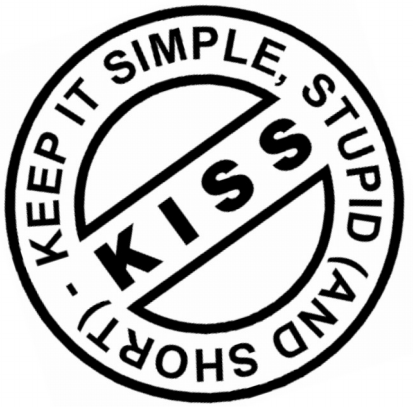


Generator (Python) 1/2

- Tutte le soluzioni precedenti tendono a “snaturare” la semplicità del problema.
- Volendo mantere il codice semplice da leggere ed efficiente??? in python potremmo fare

```
def producer():
  n = 0
  while True:
    yield n
    n += 1

def consumer():
  for data in producer():
    print data
```



- Non male no? Produttore e consumatore rimangono semplici da capire, rimanendo logicamente ben separati. Non c'è uso di modi strani per far comunicare le due attività.

  Come fa a funzionare? Semplice i due codici cooperano. Cosa vuol dire? Si esegue codice del produttore e del consumatore alternandolo in modo cooperativo.

Generator (Python) 2/2

- Usando send

```
def producer():
  c = consumer();
  n = 0
  while True:
    c.send( n );
    n += 1
```

```
def coroutine(func):
    def start(*args,**kwargs):
        cr = func(*args,**kwargs)
        cr.next()
        return cr
    return start


@coroutine
def consumer():
  while True:
    data = (yield)
    print data
```

- Per spiegazioni dettagliate ed esempi veramente interessanti www.dabeaz.com/coroutines/

Vediamo che cosa è esattamente una Coroutine

- Possiamo vedere una coroutine come una funzione che ha più punti di uscita (più di un return) e più punti di ingresso. [Donald Knuth "Subroutines are special cases of ... coroutines."]
- Avendo più punti di ingresso, non necessariamente corrispondendi con l'inizio della funzione, le coroutine devono mantenere i valori delle variabili locali tra un esecuzione e la successiva.

Bello ma è possibile solo grazie a yield??

- Per rispondere cerchiamo di capire cosa succede e come potremmo ottenere la stessa cosa in C
- La chiamata a funzione in C funziona appoggiandosi fortemente sullo stack.
- Per poter entrare ed uscire a nostro piacimento ed in qualsiasi punto di una funzione non basta un “goto” è necessario mantenere coerente lo stack.

Lo stack in C

- Il registro ESP è usato per puntare alla cima dello stack. Il registro EBP è usato per puntare alla base dello stack.
- Gli argomenti e le variabili locali di una funzione sono salvate sullo stack e possono essere sempre identificate come offset rispetto a ESP.
- Dipendentemente dall'architettura sono standardizzati quali registri una funzione chiamata può modificare e quiali invece non può modificare (o deve ripristinare prima del suo ritorno).
- I parametri passati ad una funzione sono messi sullo stack (da destra a sinistra)
- Il valore di ritorno è salvato in EAX (se sufficiente) oppure il caller implicitamente mette un parametro extra sullo stack (come primo argomento) che verrà usato come puntatore alla variabile dove salvare il valore di ritorno.
- Il chiamante mette sullo stack i parametri e chiama la subroutine (che a sua volta salva il puntatore di ritorno)
- Il chiamato salva EBP sullo stack e muove la base dello stack sul top (EBP ← ESP)
- Il chiamato prima di ritornare copia EBP su ESP (ESP ← EBP) e prende dallo stack EBP (ripristinanto lo stack originale), poi esegue ret (che recupera il puntatore di ritorno dallo stack)
- Il chiamante dovrà riprendere dallo stack di tutti i parametri precedentemente messi (o più semplicemente muovere ESP del valore corretto)
- DETTO QUESTO E' POSSIBILE MANIPOLARE LO STACK PER SALTARE AVANTI ED INDIETRO DA UN CONTESTO DI ESECUZIONE AD UN'ALTRO

Cambiamo lo stack al volo

- Manipolare lo stack è fortemente dipendente dall'architettura.
- Per fortuna lo standard POSIX prevede una serie di funzionalità per farlo in semplicità ed in modo “ragionevolmente” portabile
- `getcontext:` salva il contesto attuale
- `setcontext:` setta un contesto precedentemente salvato
- `swapcontext:` salva l'attuale contesto e salta ad un nuovo
- `makecontext:` crea un nuovo contesto
- Dalla man page è possibile avere tutte le informazioni che ci servono per imparare ad usarle

- The mcontext_t type is machine-dependent and opaque. The ucontext_t type is a structure that has at least the following fields:

```
typedef struct ucontext {
    struct ucontext *uc_link;
    sigset_t         uc_sigmask;
    stack_t          uc_stack;
    mcontext_t       uc_mcontext;
    ...
} ucontext_t;
```

- `int getcontext(ucontext_t *ucp);`
  - The function getcontext() initializes the structure pointed at by ucp to the currently active context.
- `void makecontext(ucontext_t *ucp,`
  `void (*func)(),`
  `int argc, ...);`
  - The makecontext() function modifies the context pointed to by ucp (which was obtained from a call to getcontext(3)). Before invoking makecontext(), the caller must allocate a new stack for this context and assign its address to ucp->uc_stack, and define a successor context and assign its address to ucp->uc_link.
  - When this context is later activated (using setcontext(3) or swapcontext()) the function func is called, and passed the series of integer (int) arguments that follow argc; the caller must specify the number of these arguments in argc. When this function returns, the successor context is activated. If the successor context pointer is NULL, the thread exits.
- `int swapcontext(ucontext_t *oucp, ucontext_t *ucp);`
  - The swapcontext() function saves the current context in the structure pointed to by oucp, and then activates the context pointed to by ucp.

Produttore Consumatore In C

- Vediamo una prima soluzione che usa le funzioni posix per manipolare lo stack
- …. (vediamo col debugger che salti si fa)

Esistono anche altre tecniche? Vediamone una veramente originale: http://www.chiark.greenend.org.uk/~sgtatham/coroutines.html

```
#define crBegin static int state=0; switch(state) { case 0:
#define crReturn(i,x) do { state=i; return x; case i:; } while (0)
#define crFinish }

int producer(void) {
    static int i;

    crBegin;

    i = 0;
    while(1) {
        crReturn(1, i);
        i++;
    }

    crFinish;
}
```

```
void consumer() {
  while(1) {
    int data = producer();
    printf("%d\n", data);
  }
}
```

Sembra una follia vero? Lo è però per un umano non per un compilatore. Il compilatore vala usa questa tecnica.

Complichiamo la collaborazione

- Collaboriamo con uno scheduler. Ogni task collabora con un task specifico (lo scheduler) al fine di farsi dedicare del tempo per portare avanti più operazione “contemporaneamente”
- Vantaggi? Spesso i lock non servono perché è sempre possibile sapere quando ci saranno context-switch
- Lo stack non è condiviso tra i task. Maggiore sicurezza e possibilità di implementare metodologie per stoppare solo i task che hanno avuto problemi.
- Svantaggi? Si usa una sola CPU

Scheduler Coperativo e Event-Driven programming

- Personamente ritengo la programmazione event-driven molto complessa.
- Se poi è funzionale ancora più complessa
- Tanto complessa che se non abbiamo un garbage-collector (smart-pointer, ...) è difficile uscirne
- Ma se invece riuscissimo a integrare un event-loop con uno scheduler cooperativo?
- Vediamo come

Su internet

- http://www.boost.org/doc/libs/master/libs/coroutine/doc/html/coroutine/intro.html
- http://www.chiark.greenend.org.uk/~sgtatham/coroutines.html
- http://www.dabeaz.com/coroutines/
- http://libevent.org/
- http://swtch.com/libtask/
- https://code.google.com/p/libconcurrency/
- http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3708.pdf
- http://en.wikipedia.org/wiki/Coroutine
- https://docs.python.org/3/library/asyncio-task.html